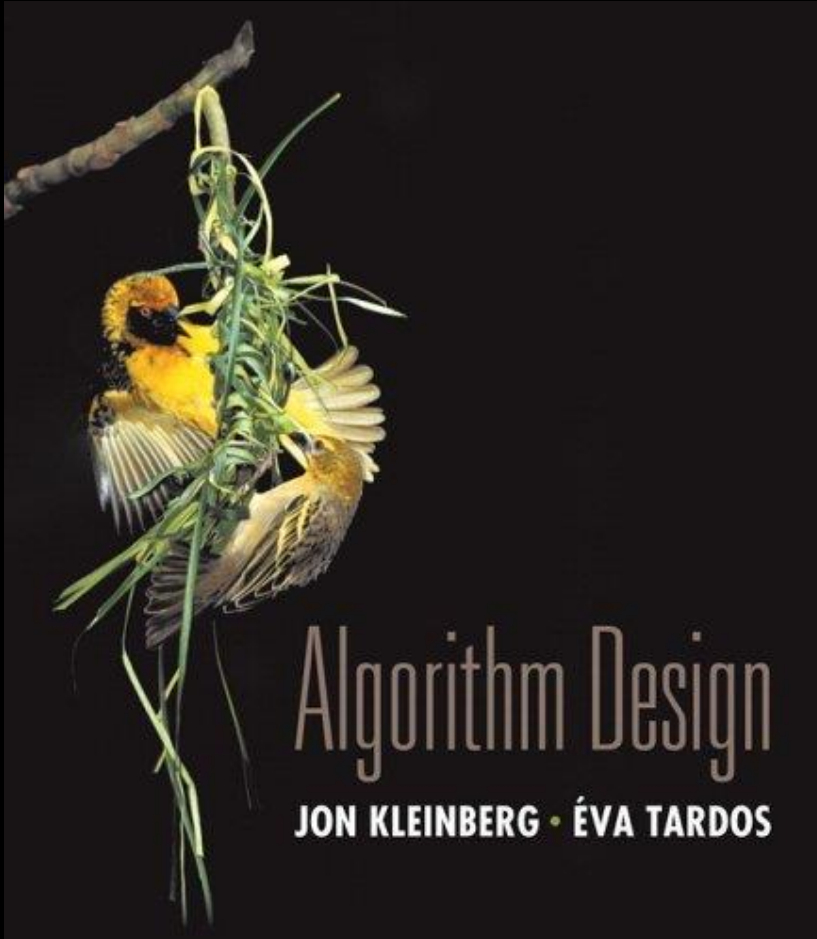


Chapter 4

Greedy Algorithms

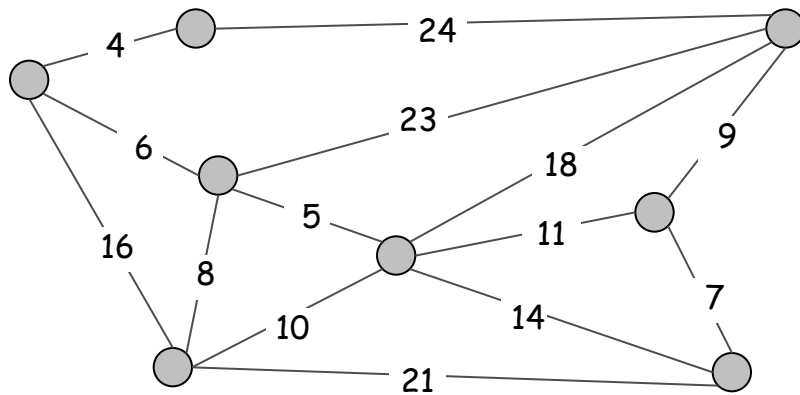


Slides by Kevin Wayne.
Copyright © 2005 Pearson-Addison Wesley.
All rights reserved.

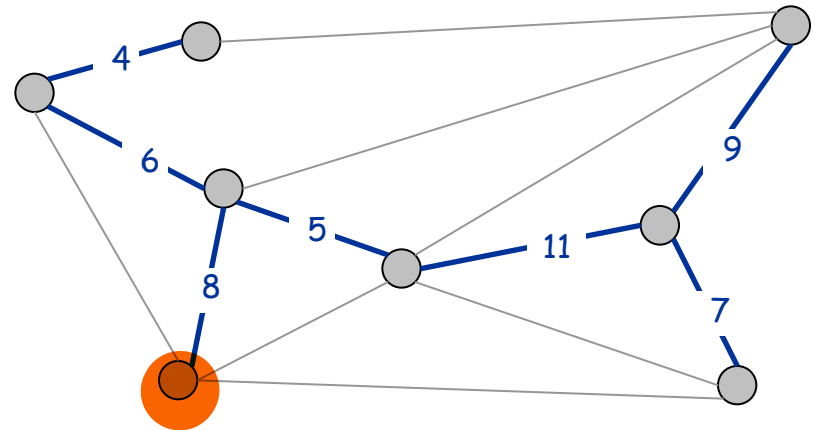
4.5 Minimum Spanning Tree

Minimum Spanning Tree

Minimum spanning tree. Given a connected undirected graph $G = (V, E)$ with real-valued edge weights c_e , an **MST** is a subset of the edges $T \subseteq E$ such that T is a spanning tree whose sum of edge weights is minimized.



$G = (V, E)$



$T, \sum_{e \in T} c_e = 50$

TUTTI I COLLEGAMENTI
CON COSTO INF.

Cayley's Theorem. There are n^{n-2} spanning trees of K_n .

can't solve by brute force

The Minimum Spanning Tree (MST) problem

- **Input:**
 - a connected weighted undirected graph $G = (V, E)$ with real-valued edge weights c_e
- **Feasible solution:**
 - a spanning tree T of G , i.e. a tree with $T \subseteq E$ reaching all vertices of G
- **measure (to minimize):**
 - the weight (or cost) of T , i.e. $c(T) = \sum_{e \in T} c_e$

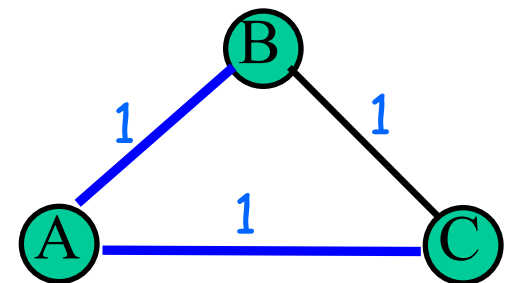
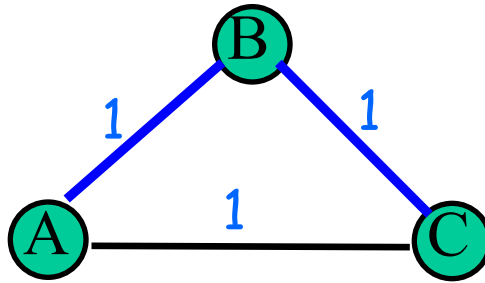
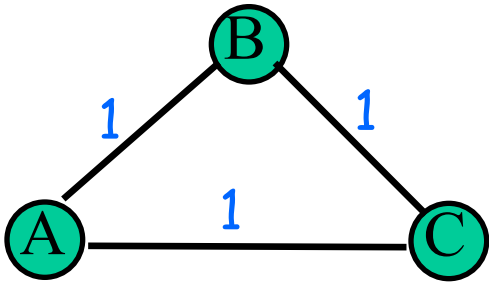
Applications

MST is fundamental problem with diverse applications.

- Network design.
 - telephone, electrical, hydraulic, TV cable, computer, road
- Approximation algorithms for NP-hard problems.
 - traveling salesperson problem, Steiner tree
- Indirect applications.
 - max bottleneck paths
 - LDPC codes for error correction
 - image registration with Renyi entropy
 - learning salient features for real-time face verification
 - reducing data storage in sequencing amino acids in a protein
 - model locality of particle interactions in turbulent fluid flows
 - autoconfig protocol for Ethernet bridging to avoid cycles in a network
- Cluster analysis.

Uniqueness of MST

The MST is not unique in general



Property: If G has distinct weights then the MST is unique.

exercise: prove it.

Greedy Algorithms

Kruskal's algorithm. Start with $T = \emptyset$. Consider edges in ascending order of cost. Insert edge e in T unless doing so would create a cycle.

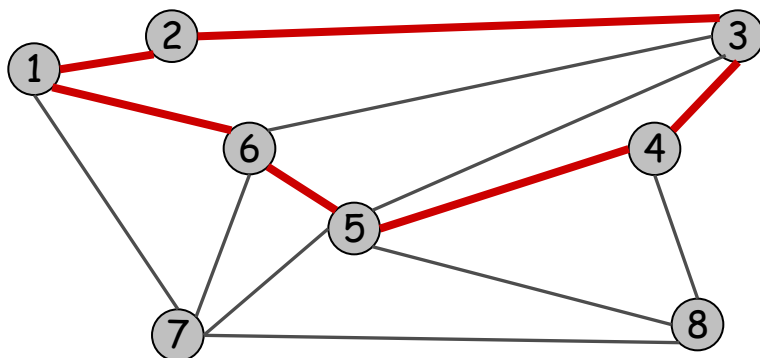
 **Reverse-Delete algorithm.** Start with $T = E$. Consider edges in descending order of cost. Delete edge e from T unless doing so would disconnect T .

Prim's algorithm. Start with some root node s and greedily grow a tree T from s outward. At each step, add the cheapest edge e to T that has exactly one endpoint in T .

Remark. All three algorithms produce an MST.

Cycles and Cuts

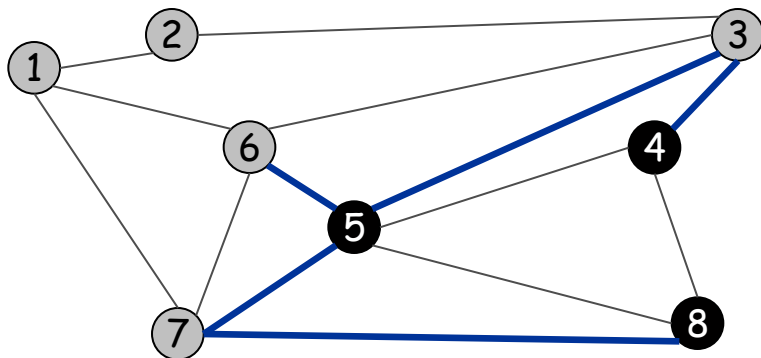
Cycle. Set of edges the form a-b, b-c, c-d, ..., y-z, z-a.



Cycle $C = 1-2, 2-3, 3-4, 4-5, 5-6, 6-1$

A Cut. A cut is a subset of nodes S . (Sometime defined as a partition of V into S and $V \setminus S$.)

Cutset. The corresponding cutset D of a cut S is the subset of edges with exactly one endpoint in S .

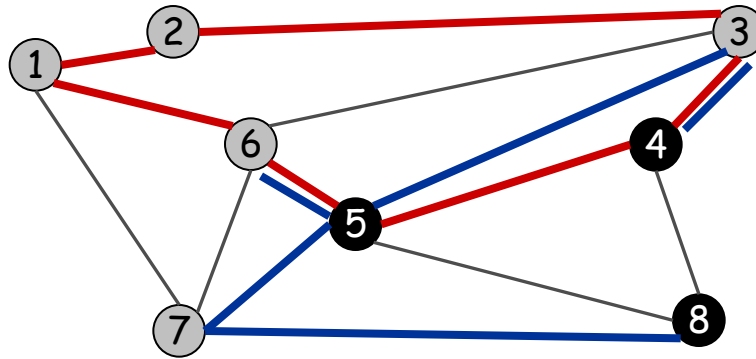


SONO TUTTI QUEGLI ARCHI
CHE HANNO nodes in S
e uno non
GLI UNZIONE

Cut $S = \{4, 5, 8\}$
Cutset $D = 5-6, 5-7, 3-4, 3-5, 7-8$

Cycle-Cut Intersection

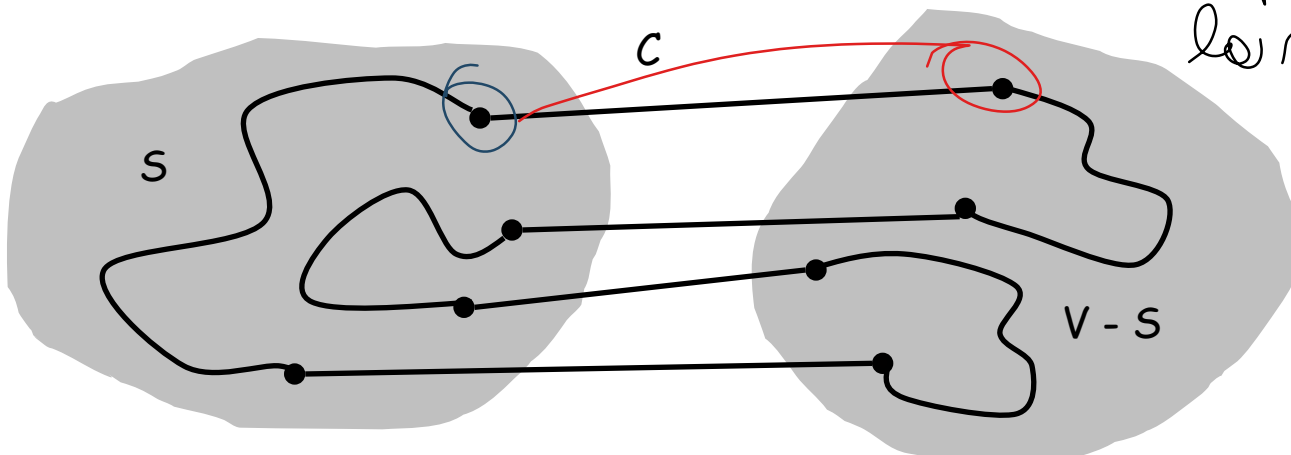
Claim. A cycle and a cutset intersect in an **even** number of edges. ARCHI



Cycle $C = 1-2, 2-3, 3-4, 4-5, 5-6, 6-1$
 Cutset $D = 3-4, 3-5, 5-6, 5-7, 7-8$
 Intersection = $3-4, 5-6$

Pf. (by picture)

*il fatto che devo
 tornare indietro
 lo rende
 pari*

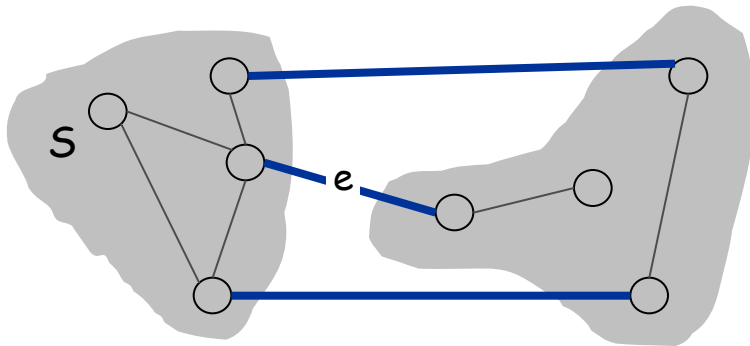


Greedy Algorithms

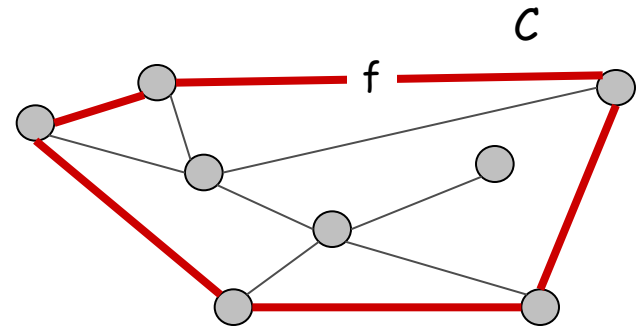
Cut property. Let S be any subset of nodes, and let e be a min cost edge with exactly one endpoint in S . Then there exists an MST containing e .

$\uparrow$ min cut $S \in T$

Cycle property. Let C be any cycle, and let f be a max cost edge belonging to C . Then there exists an MST that does not contain f .



e is in some MST



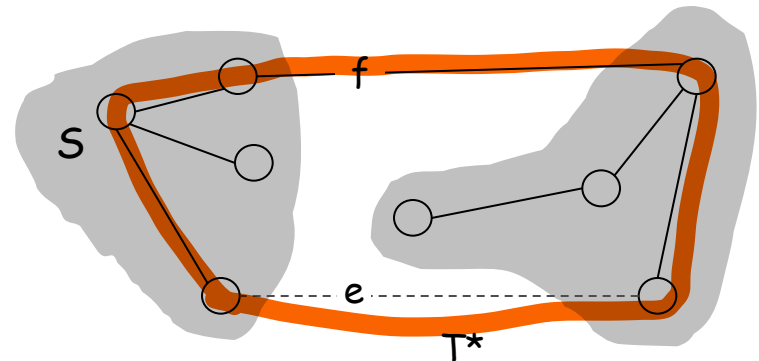
f is not in some MST

Greedy Algorithms

Cut property. Let S be any subset of nodes, and let e be a min cost edge with exactly one endpoint in S . Then there exists an MST T^* containing e .

Pf. (exchange argument) $\leftarrow \{e\} - \{f\}$

- Suppose e does not belong to T^* .
- Adding e to T^* creates a cycle C in T^* . ●
- Edge e is both in the cycle C and in the cutset D corresponding to S
essendo ciclo
 $\Rightarrow$ there exists another edge, say f , that is in both C and D .
- $T' = T^* \cup \{e\} - \{f\}$ is also a spanning tree.
- Since $c_e \leq c_f$, $\text{cost}(T') \leq \text{cost}(T^*)$.
- Then T' is an **MST** containing e . ■

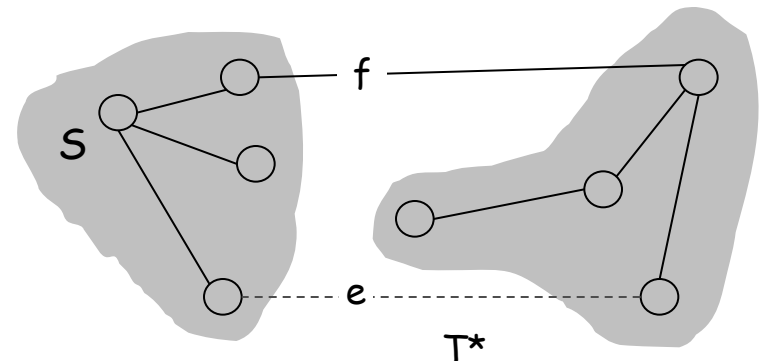


Greedy Algorithms

Cycle property. Let C be any cycle in G , and let f be a max cost edge belonging to C . Then there exists an MST T^* that does not contain f .

Pf. (exchange argument)

- Suppose f belongs to T^* .
- Deleting f from T^* creates a cut S in T^* .
- Edge f is both in the cycle C and in the cutset D corresponding to S
 $\Rightarrow$ there exists another edge, say e , that is in both C and D .
- $T' = T^* \cup \{e\} - \{f\}$ is also a spanning tree.
- Since $c_e \leq c_f$, $\text{cost}(T') \leq \text{cost}(T^*)$.
- Then T' is an MST that does not contain f . ■



Kruskal's algorithm

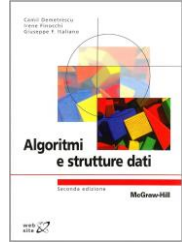
Kruskal's algorithm. Start with $T = \phi$. Consider edges in ascending order of cost. Insert edge e in T unless doing so would create a cycle.

Remark.

An efficient implementation of **Kruskal's algorithm** uses a **Union-Find** data structure:

- to maintain the connected components of the current solutions
- to check whether the current edge forms a cycle (with the current solution)

QUICK UNION
QUICK FIND
STANDARD



algorithm Kruskal (graph $G=(V,E,c)$)

UnionFind UF

$T=\emptyset$

sort the edges in ascending order of costs

for each vertex v **do** UF.makeset(v)

for each edge (x,y) in order **do**

T_x =UF.find(x)

T_y =UF.find(y)

if $T_x \neq T_y$ **then**

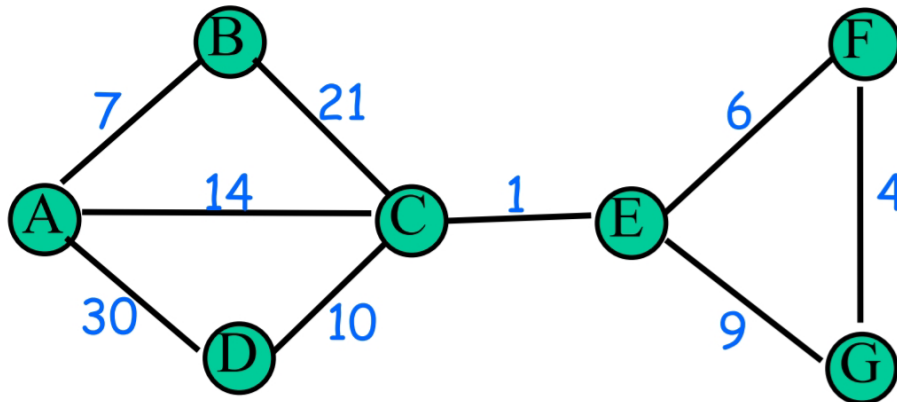
UF.union(T_x, T_y)

add edge (x,y) to T

return T

SE SONO DUE RADICI
DIVERSE

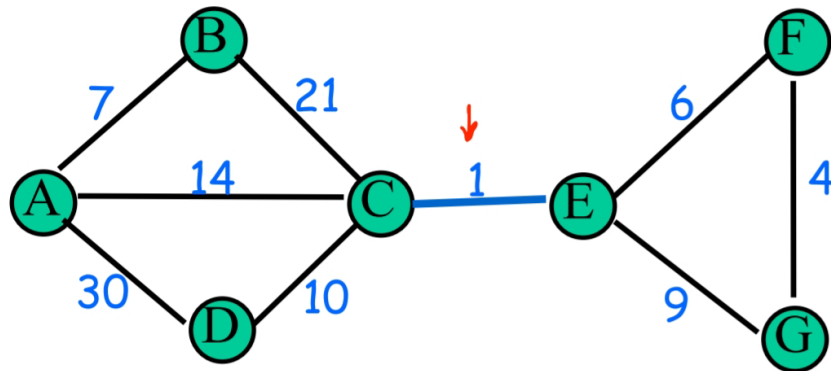
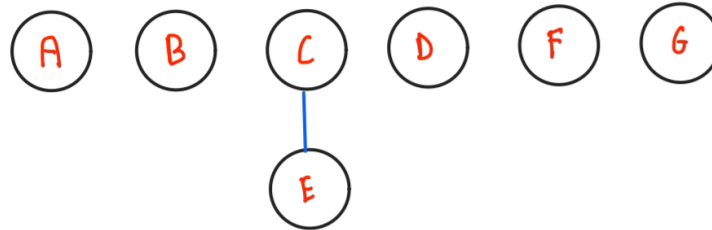
Immagina di aver fatto i ~~max~~set



$\text{Find}(C)=C$
 $\text{Find}(E)=E$ } $\neq$

union (C, E)

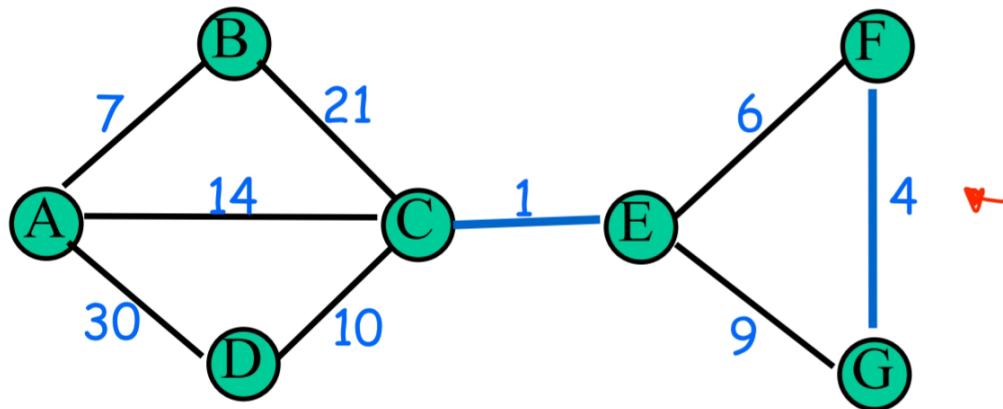
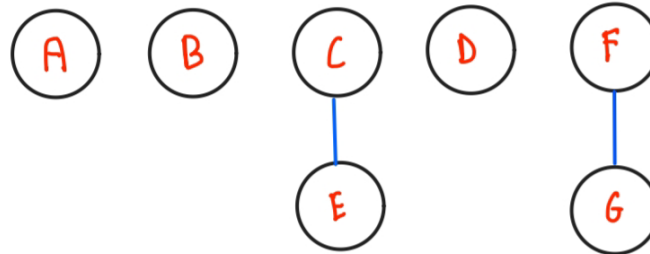
$T = \{(C, E)\}$



$\text{Find}(F) = F$
 $\text{Find}(G) = G$

union(F, G)

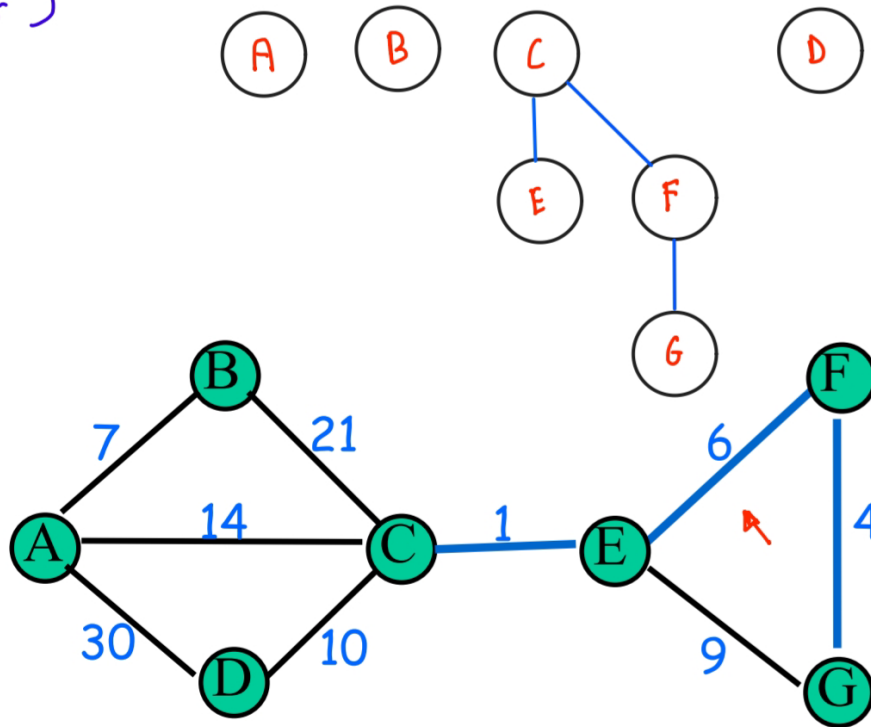
$T = \{(C-E), (F-G)\}$



$\left. \begin{array}{l} \text{Find}(E) = C \\ \text{Find}(F) = F \end{array} \right\} \neq$

$\text{union}(E, F)$

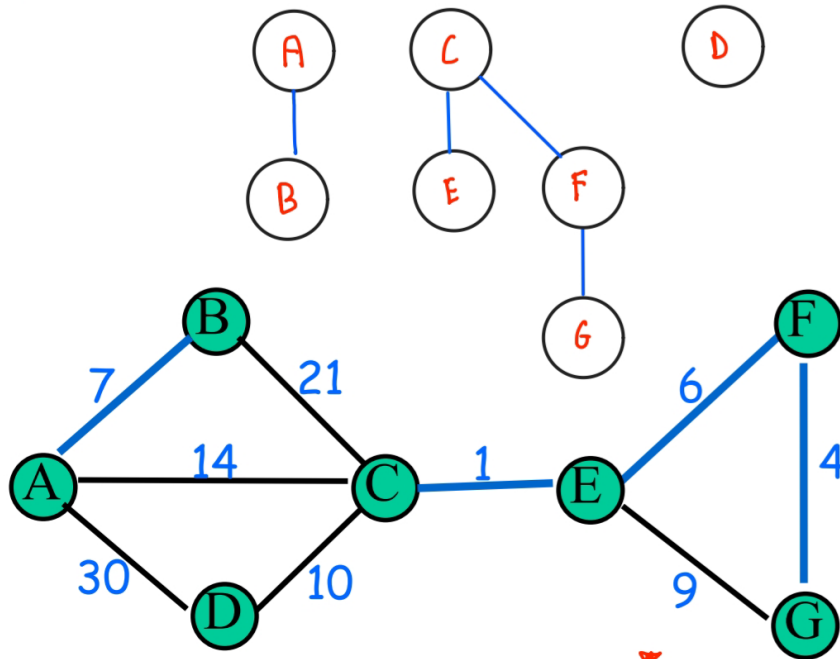
$T = \{(C, E), (F, G), (E, F)\}$



$\text{Find}(A) = A$
 $\text{Find}(B) = B$ } $\neq$

Union (A, B)

$T = \{(C, E), (F, G), (E, F), (A, B)\}$

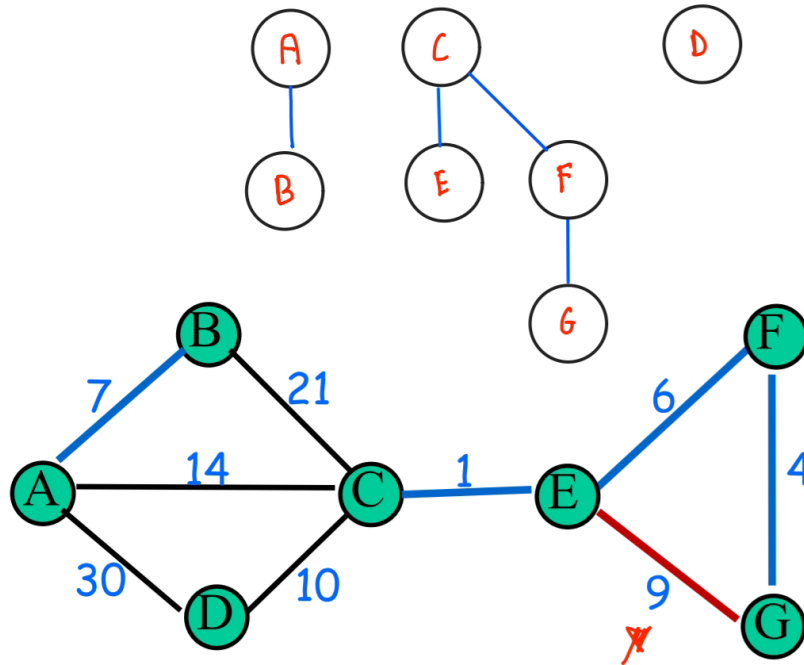


$\text{find}(E) = \text{find}(G)$

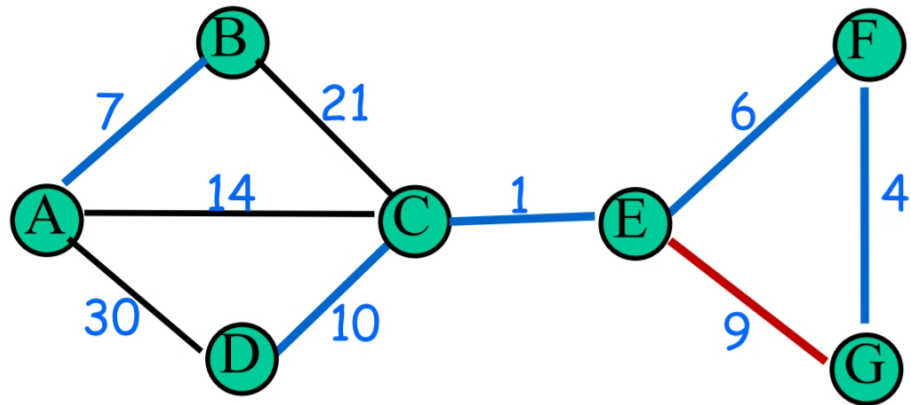
NON metto l'arco (E, G)

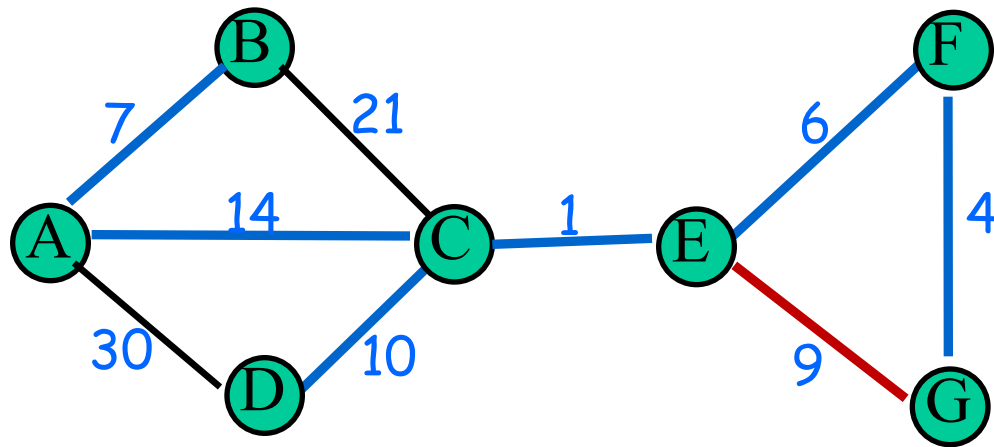
$\left. \begin{array}{l} \text{Find}(A) = C \\ \text{Find}(E) = C \end{array} \right\} = \text{NON FACCI O union}$

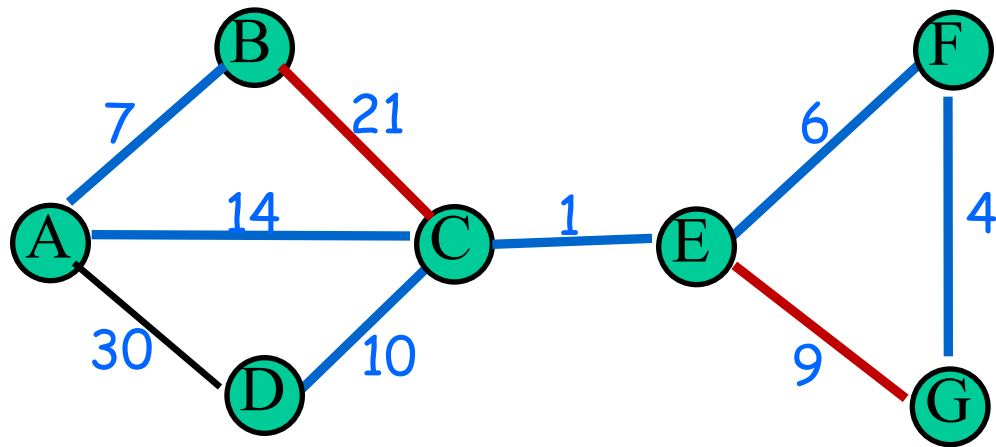
$T = \{(C-E), (F-G), (E, F), (A, B)\}$

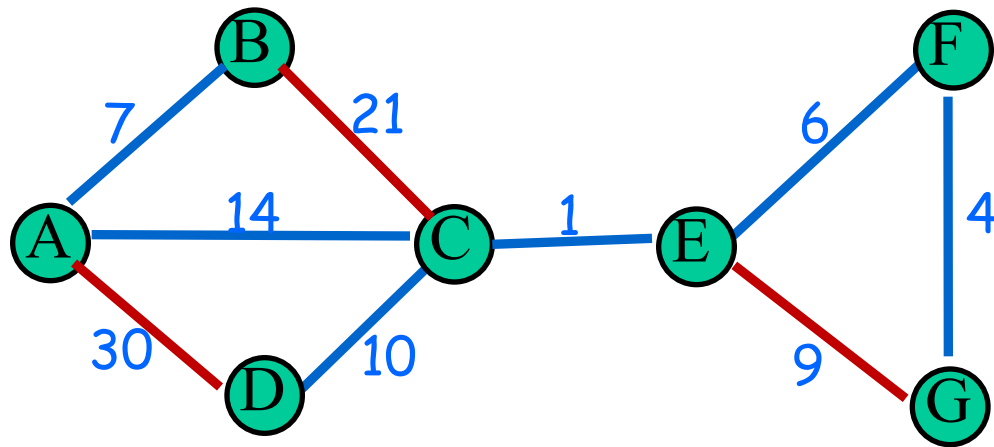


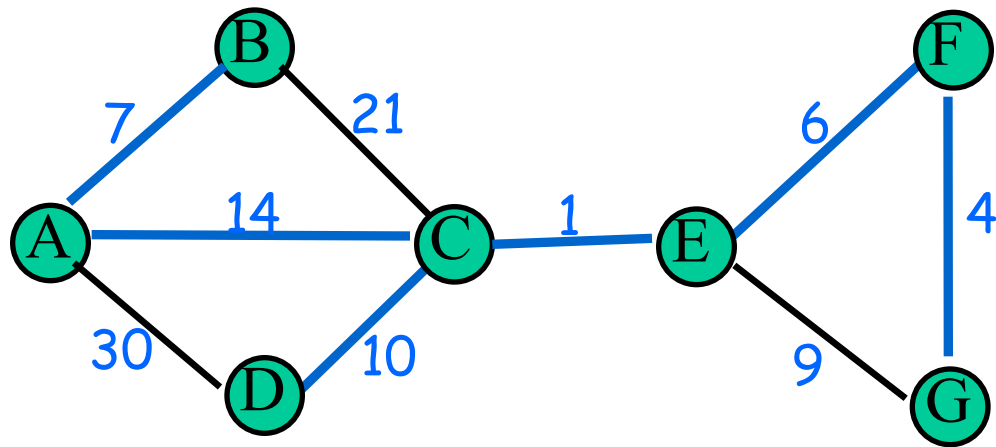
e così via...





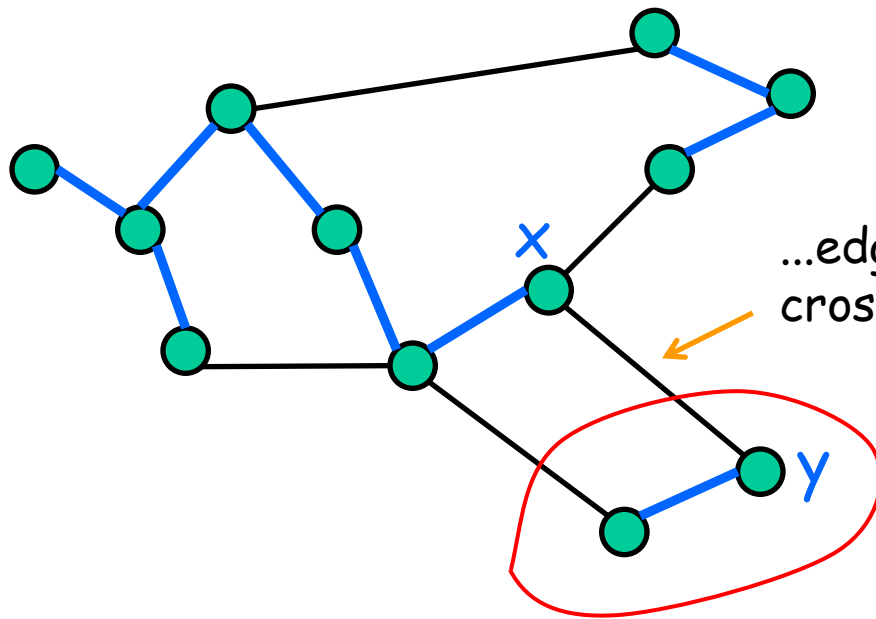






Correctness of Kruskal's algorithm

when the algorithm decides to add edge (x,y) to the solution



since the algorithm look at the edges in increasing order of costs...

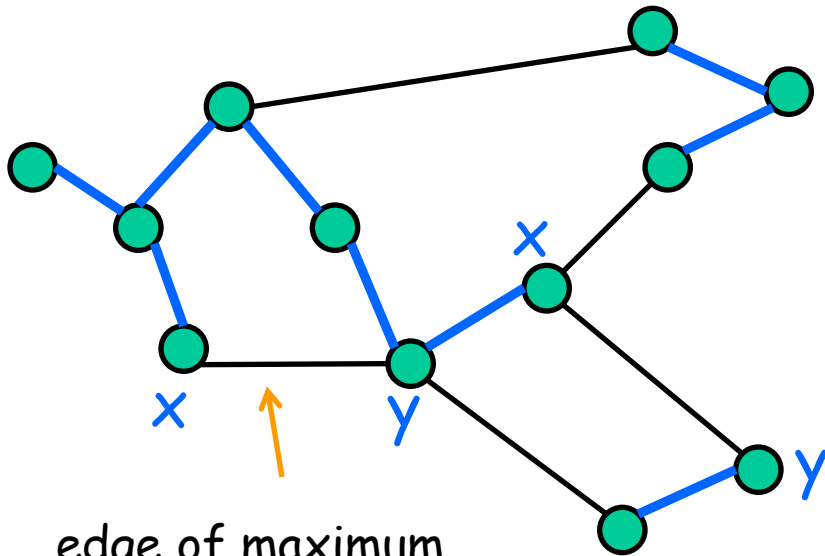
...edge of minimum cost crossing the cut $S, V \setminus S$

consider the set S of vertices belonging to the same connected component of y

NON CREO
UN CICLO
QUINDI
PRENDO

Correctness of Kruskal's algorithm

when the algorithm decides to kick out edge (x,y) from the solution



...edge of maximum weight in the cycle that (x,y) forms with the current solution

since the algorithm look at the edges in increasing order of costs...

CREUN
CICLO
NONLO
PRENDO

Running time of Kruskal's algorithm

- sorting the edges: $O(m \log m) = O(m \log n)$

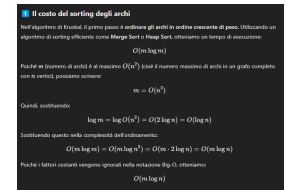
- Union-Find operations:

-n makeset ops

-n-1 union ops

-m find ops

Ricorda
n nodi
m archi
m è $O(n^2)$ ma
è anche un $O(n^2)$



algorithm Kruskal (graph $G=(V,E,c)$)

UnionFind UF

$T=\emptyset$

sort the edges in ascending order of costs

for each vertex v **do** UF.makeset(v)

for each edge (x,y) in order **do**

T_x =UF.find(x)

T_y =UF.find(y)

if $T_x \neq T_y$ **then**

UF.union(T_x, T_y)

add edge (x,y) to T

return T

➡ $O(m \log n + UF(m,n))$

-using QuickFind with union by size

$O(m \log n + m + n \log n) = O(m \log n)$

-using QuickUnion with union by size

$O(m \log n + m \log n + n) = O(m \log n)$

$O(m \log n)$

Prim's algorithm

The Minimum Spanning Tree (MST) problem

- Input:

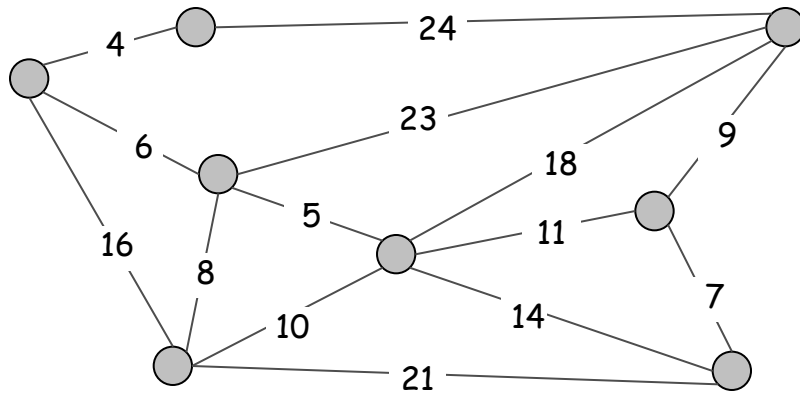
- a connected weighted undirected graph $G = (V, E)$ with real-valued edge weights c_e

- Feasible solution:

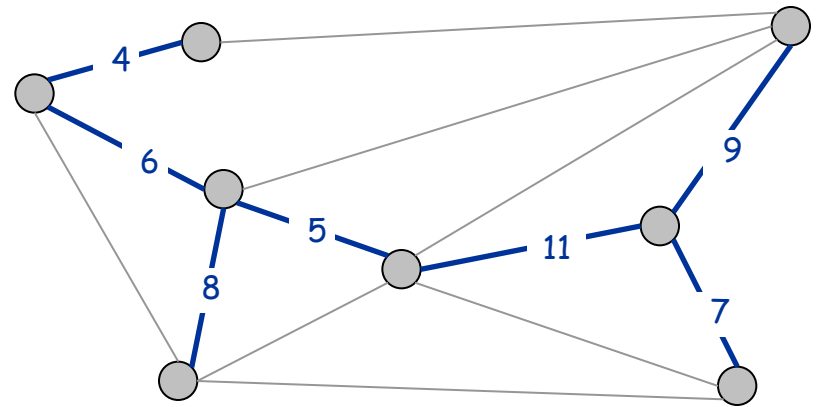
- a spanning tree T of G , i.e. a tree $T=(V,F)$ with $T \subseteq E$ (reaching all vertices of G)

- measure (to minimize):

- the weight (or cost) of T , i.e. $c(T) = \sum_{e \in T} c_e$



$G = (V, E)$

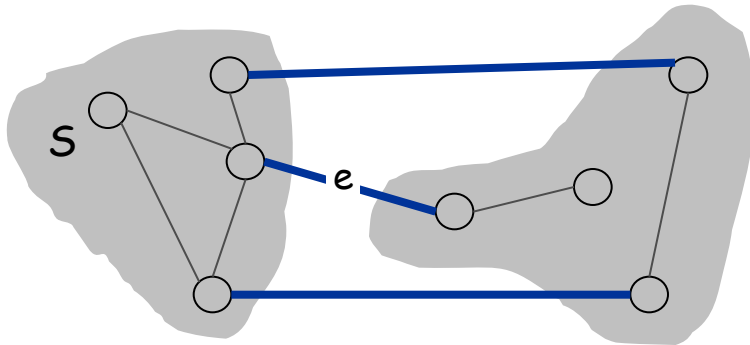


$T, \sum_{e \in T} c_e = 50$

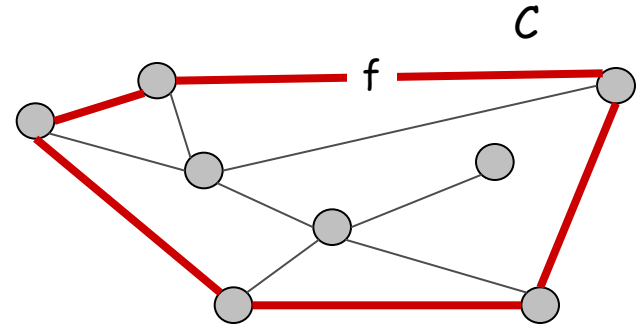
Greedy Algorithms

Cut property. Let S be any subset of nodes, and let e be a min cost edge with exactly one endpoint in S . Then there exists an MST containing e .

Cycle property. Let C be any cycle, and let f be a max cost edge belonging to C . Then there exists an MST that does not contain f .



e is in some MST



f is not in some MST

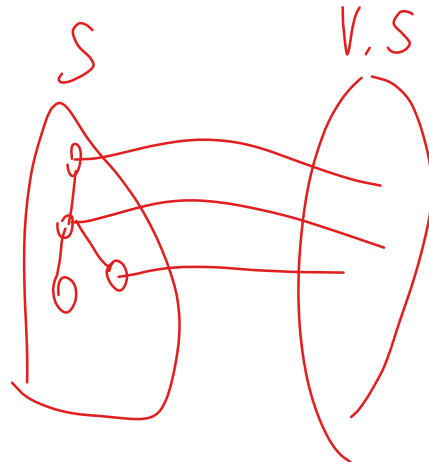
Prim's algorithm [Jarník 1930, Dijkstra 1957, Prim 1959].

Start with some root node s and greedily grow a tree T from s outward. At each step, add the cheapest edge e to T that has exactly one endpoint in T .

Correctness.

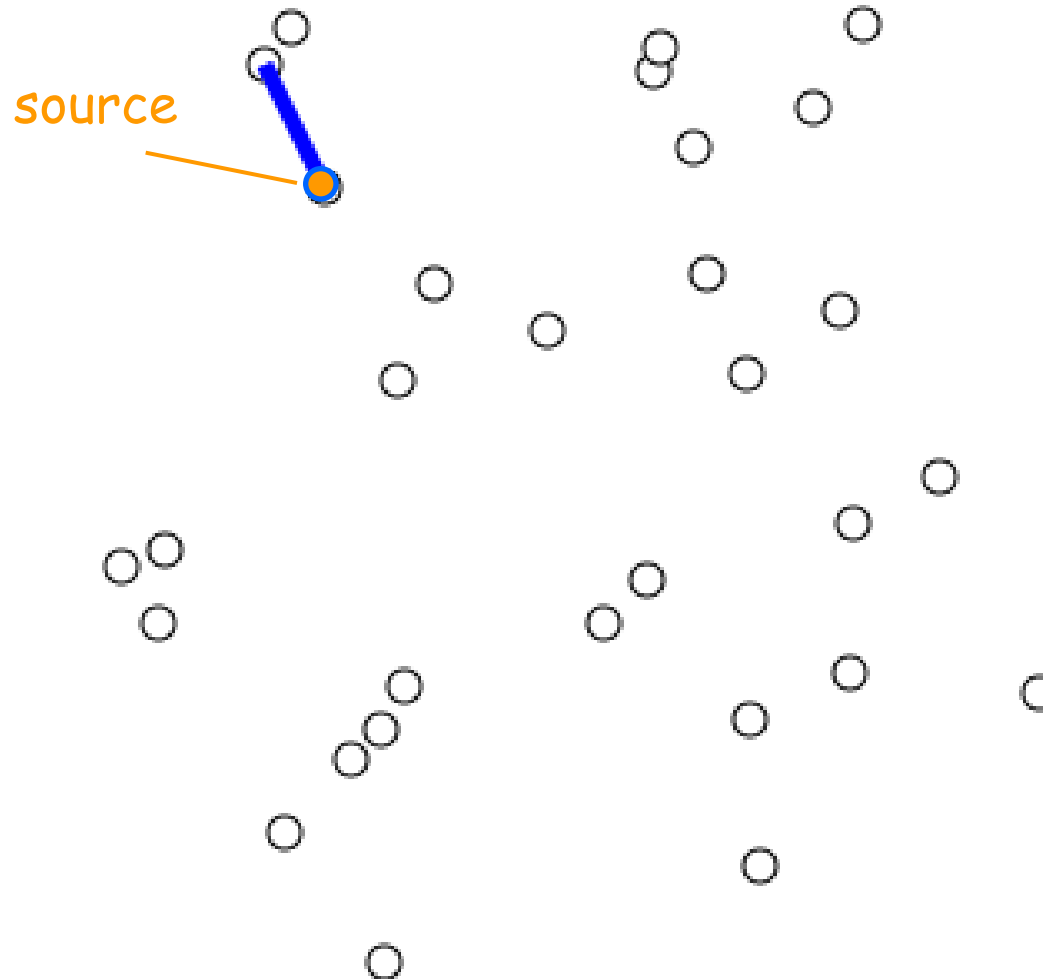
Immediate consequence of the cut property, used exactly $n-1$ times.

now continue
A

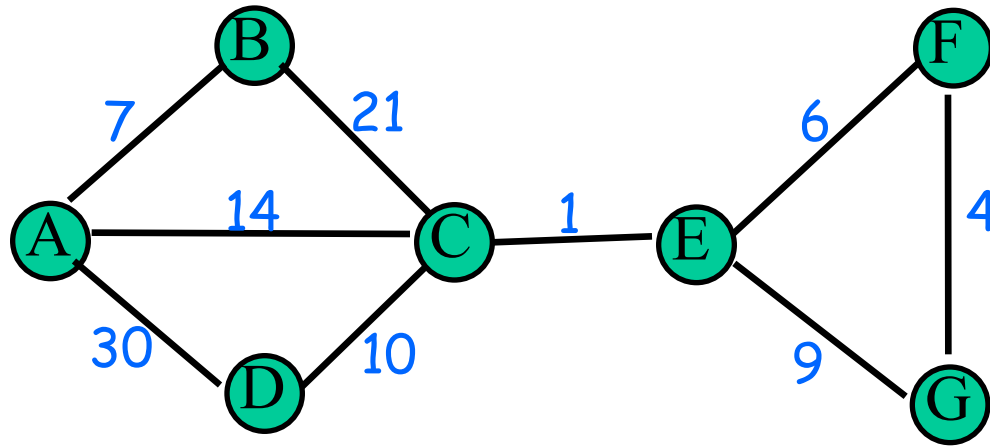


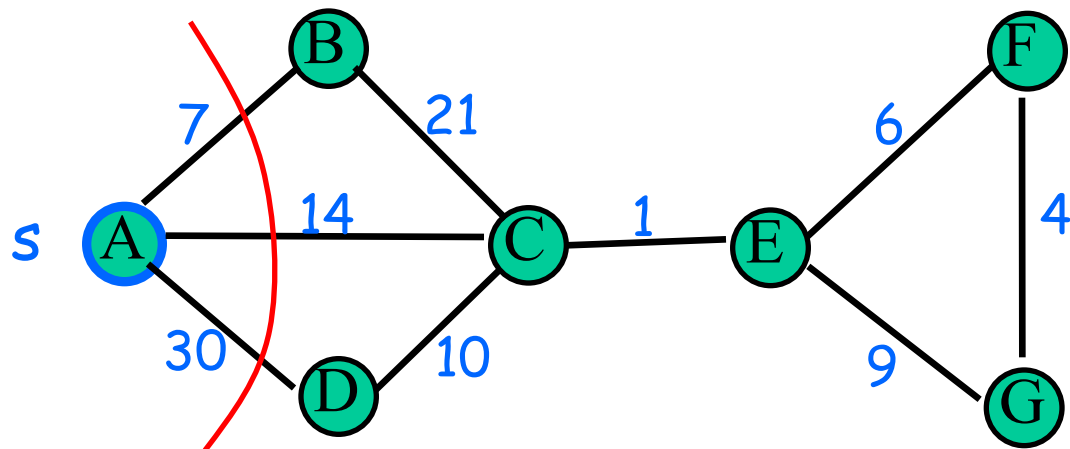
Euclidean complete graph

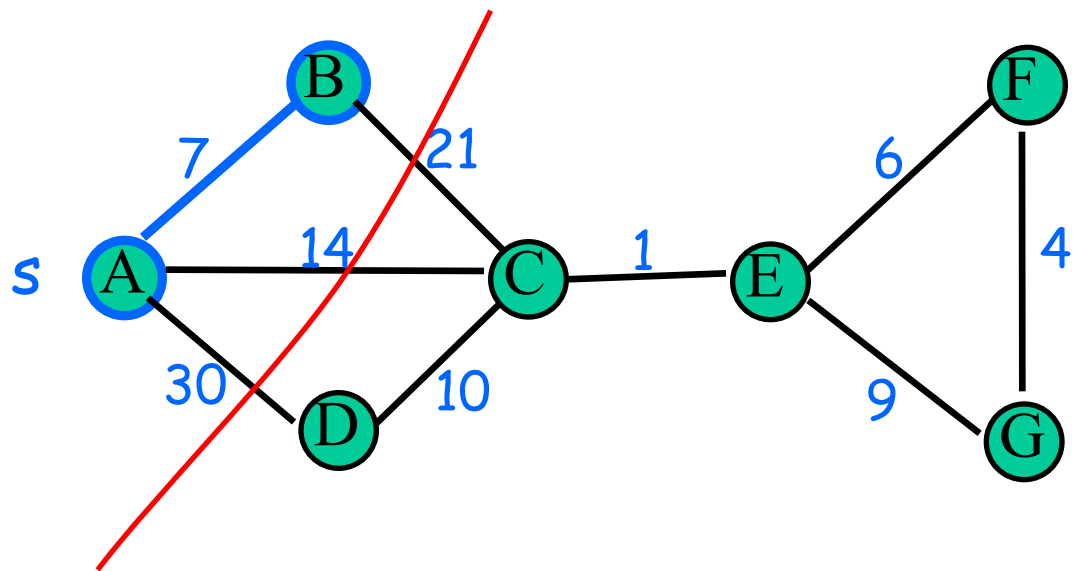
- vertices placed on the plane
- for each pair of vertices u and v the cost of edge (u,v) is the Euclidean distance between u and v .

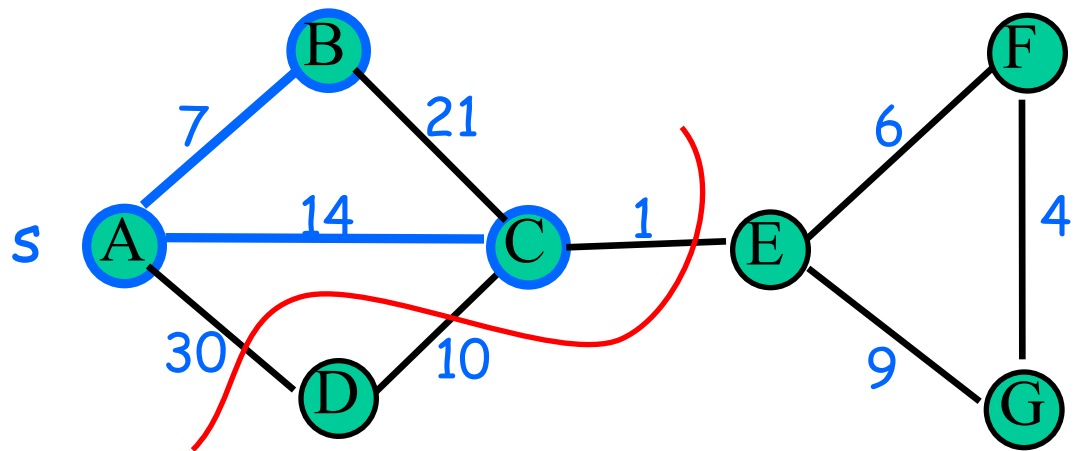


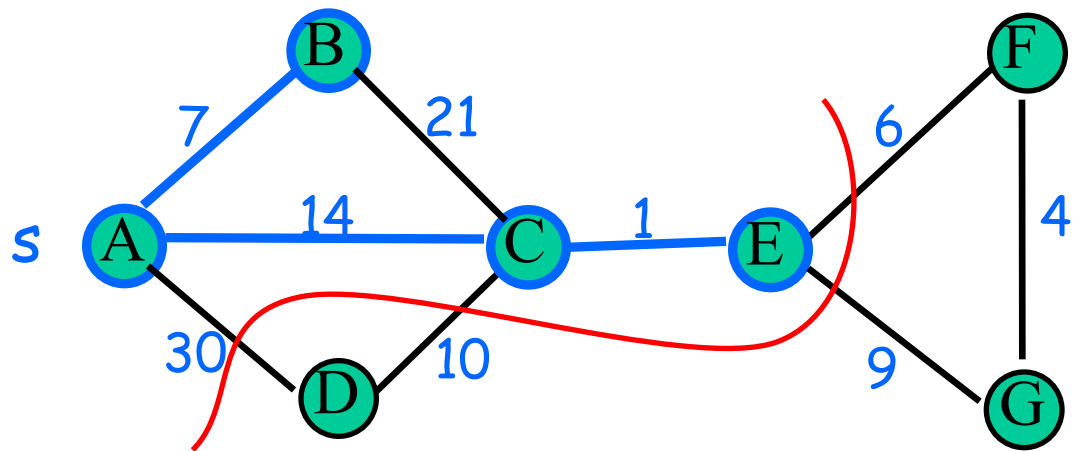
IDEA: partire da un nodo (in questo caso A) crea il mio MST controllando
ogni arco di ogni
cutset

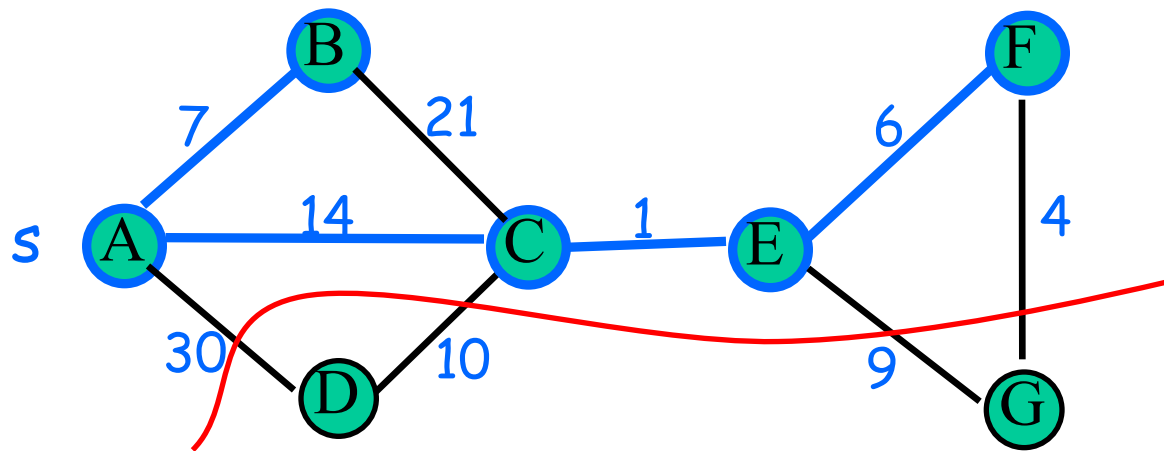


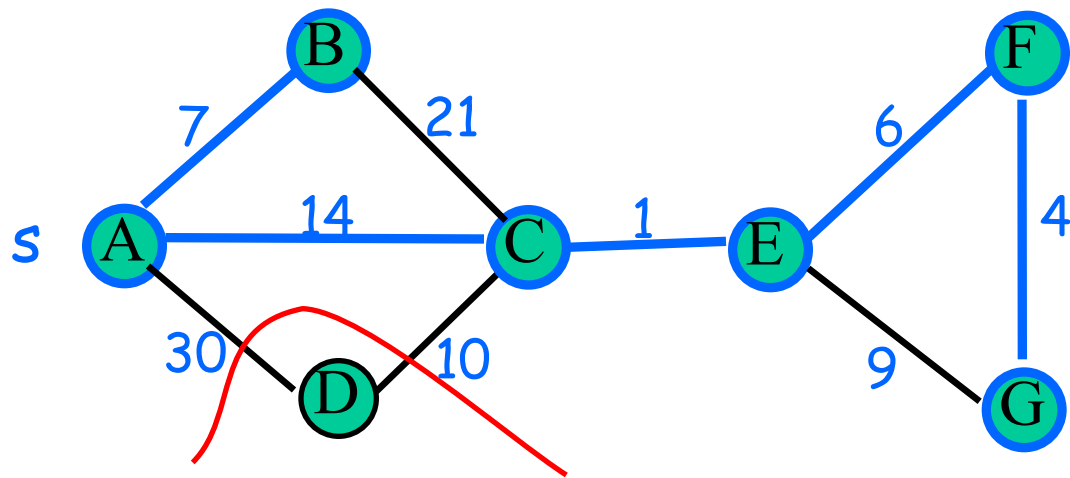


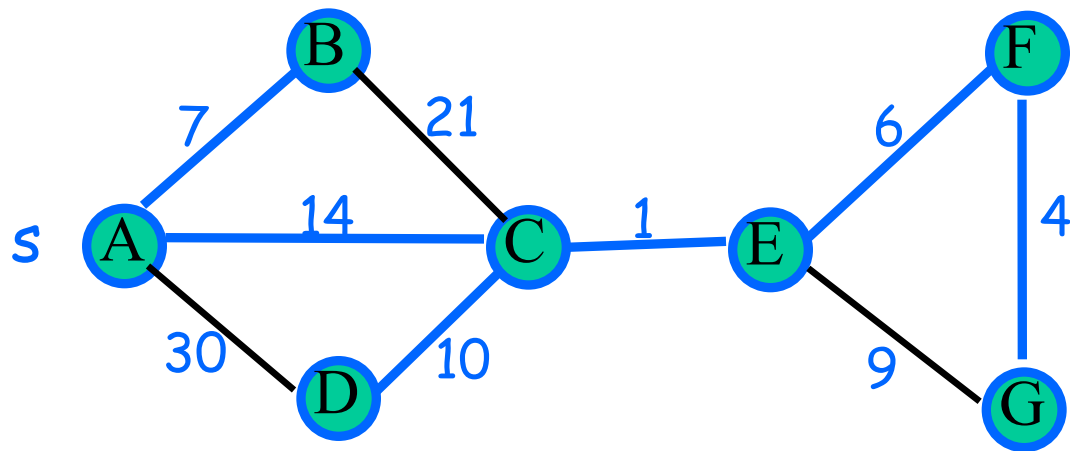












Running time

A simple (and inefficient) implementation:

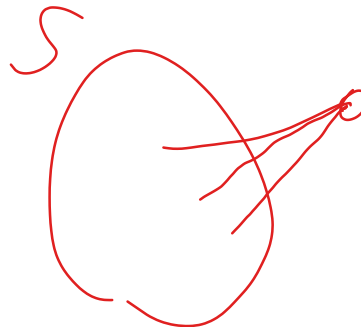
For $n-1$ times, find a **cheapest** edge crossing the cut induced by the current partial tree in $O(m)$ time.

Total running time: **$O(mn)$** .

A much faster implementation:

simile algoritmo Dijkstra

- Maintain set of explored nodes S .
- **Use a priority queue** to maintain unexplored nodes.
- For each unexplored node v , the priority is the attachment cost $a[v]$ = cost of a cheapest edge incident in v having the other endpoint in S .



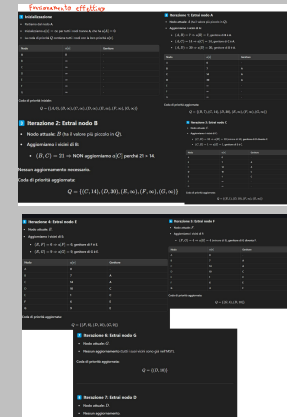
```

Prim(G, s) {
  foreach (v ∈ V) a[v] ← ∞
  a[s] ← 0
  Initialize an empty priority queue Q
  foreach (v ∈ V) insert v onto Q with priority a[v]
  Initialize set of explored nodes S ← ∅
  Initialize T to the tree containing only s.

  while (Q is not empty) {
    u ← delete min element from Q
    S ← S ∪ { u }
    foreach (edge e = (u, v) incident to u)
      if ((v ∉ S) and (ce < a[v]))
        make u parent of v in T
        decrease priority a[v] to ce

  return T
}

```



Running time.

- ◻ $O(m+n)$ time plus the cost of the priority queue operations
- ◻ n inserts, n delete min ops, $2m$ decrease key ops
- ◻ $O(n^2)$ with an array; $O(m \log n)$ with a binary heap;
 $O(m + n \log n)$ with Fibonacci's heaps

```

Prim(G, s) {
  foreach ( $v \in V$ )  $a[v] \leftarrow \infty$ 
   $a[s] \leftarrow 0$ 
  Initialize an empty priority queue Q
  foreach ( $v \in V$ ) insert  $v$  onto Q with priority  $a[v]$ 
  Initialize set of explored nodes  $S \leftarrow \emptyset$ 
  Initialize T to the tree containing only s.

  while (Q is not empty) {
     $u \leftarrow$  delete min element from Q
     $S \leftarrow S \cup \{u\}$ 
    foreach (edge  $e = (u, v)$  incident to  $u$ )
      if ( $(v \notin S)$  and ( $c_e < a[v]$ ))
        make  $u$  parent of  $v$  in T
        decrease priority  $a[v]$  to  $c_e$ 
  }
  return T
}

```

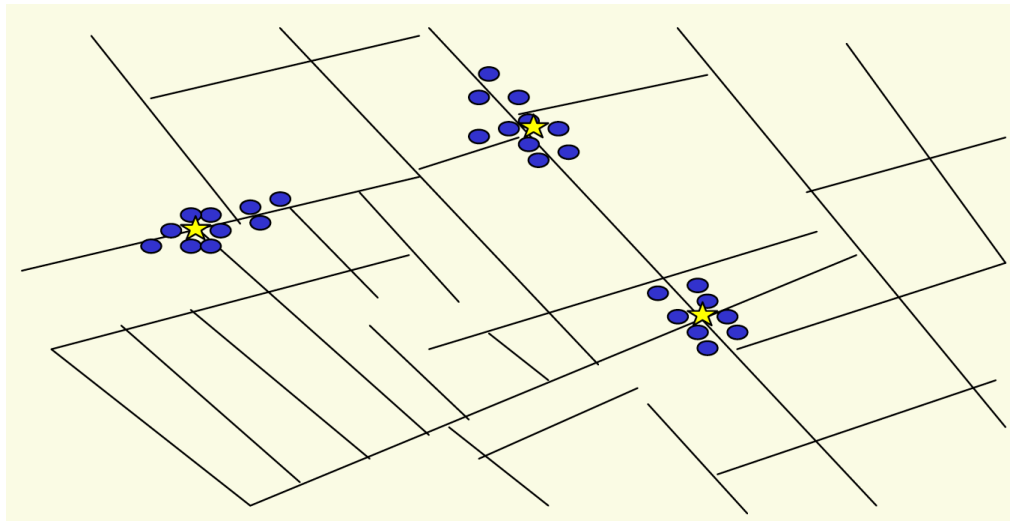
Running time.

- $O(m+n)$ time plus the cost of priority queue operations
- n inserts, n delete min operations
- $O(n^2)$ with an array; $O(m + n \log n)$ with Fibonacci's heaps

$$O(m + n \log n)$$

4.7 Clustering

Raccolta in gruppi di oggetti con una logica



Outbreak of cholera deaths in London in 1850s.
Reference: Nina Mishra, HP Labs

Clustering

Clustering. Given a set U of n objects labeled $p_1, \dots, p_n$, classify into coherent groups.

↑
photos, documents, micro-organisms

Distance function. Numeric value specifying "closeness" of two objects.

↑
number of corresponding pixels whose intensities differ by some threshold

Vicinanza geografica oppure per altri motivi

come nelle foto piu ci sono pixel simili significa che sono "vicine" viceversa "lontane"

Fundamental problem. Divide into clusters so that points in different clusters are far apart.

- Routing in mobile ad hoc networks.
- Identify patterns in gene expression.
- Document categorization for web search.
- Similarity searching in medical image databases
- Skycat: cluster 10^9 sky objects into stars, quasars, galaxies.

Clustering of Maximum Spacing

k-clustering. Divide objects into k non-empty groups.

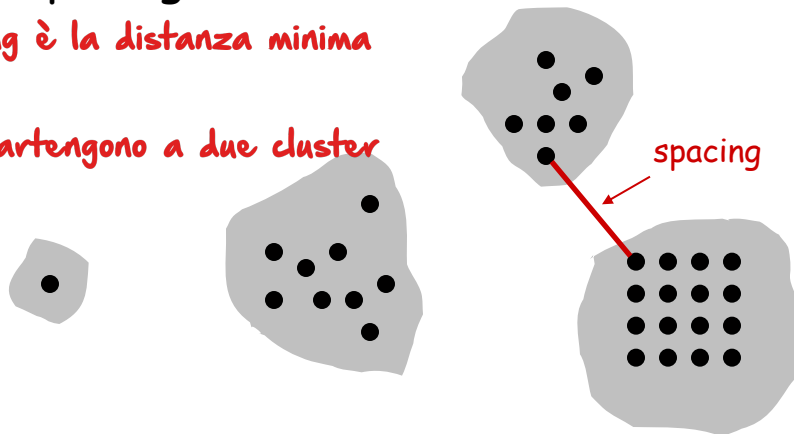
Distance function. Assume it satisfies several natural properties.

- $d(p_i, p_j) = 0$ iff $p_i = p_j$ (identity of indiscernibles) *Se distanza 0 l'oggetto è se stesso*
- $d(p_i, p_j) \geq 0$ (nonnegativity) *no negative*
- $d(p_i, p_j) = d(p_j, p_i)$ (symmetry) *distanza uguale*

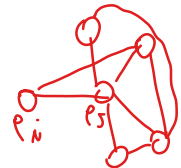
Spacing. Min distance between any pair of points in different clusters.

Clustering of maximum spacing. Given an integer k , find a k -clustering of maximum spacing.

spacing di un clustering è la distanza minima che intercorre tra due punti che appartengono a due cluster diversi



ES:



$k = 4$

quindi tipo mettere tutti i cluster vicini in modo che la distanza tra i singoli punti sia il minimo? ?

Greedy Clustering Algorithm

all'inizio ho tutti i cluster separati

Single-linkage k-clustering algorithm.

- Form a graph on the vertex set U , corresponding to n clusters.
- Find the closest pair of objects such that each object is in a different cluster, and add an edge between them (and merge the corresponding clusters).

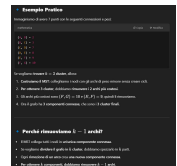
- Repeat $n-k$ times until there are exactly k clusters.

trova lo spacing + piccolo tra due oggetti e li unisco in un solo cluster

Key observation. This procedure is precisely Kruskal's algorithm (except we stop when there are k connected components).

a quanto pare ricorda l'algoritmo di kruskal

Remark. Equivalent to finding an MST and deleting the $k-1$ most expensive edges.



Hierarchical Clustering. Running Kruskal's algorithm until the end implicitly produces a Hierarchical Clustering, i.e. a k -clustering for each value of $k=n, n-1, \dots, 1$.

visto che potresti non sapere quale k cerchi provi a farlo per ogni k giusto finché non trovi il MST



	BOS	NY	CHI	DEN	SF	SEA
BOS	0	206	963	1949	3095	2979
NY		0	802	1771	2934	2815
CHI			0	966	2142	2013
DEN				0	1235	1307
SF					0	808
SEA						0

Single linkage

$k=6$

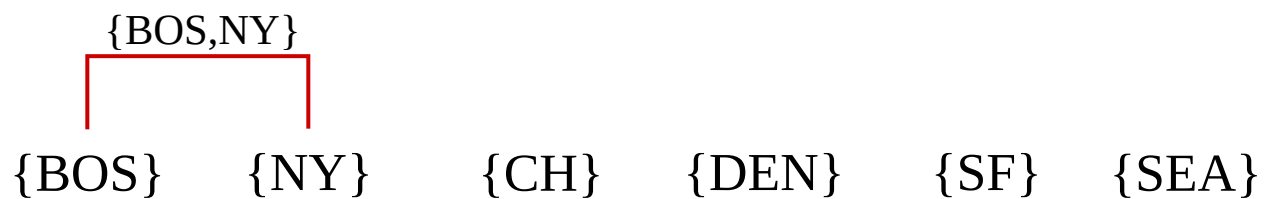
{BOS} {NY} {CH} {DEN} {SF} {SEA}



	BOS	NY	CHI	DEN	SF	SEA
BOS	0	206	963	1949	3095	2979
NY		0	802	1771	2934	2815
CHI			0	966	2142	2013
DEN				0	1235	1307
SF					0	808
SEA						0

Single linkage

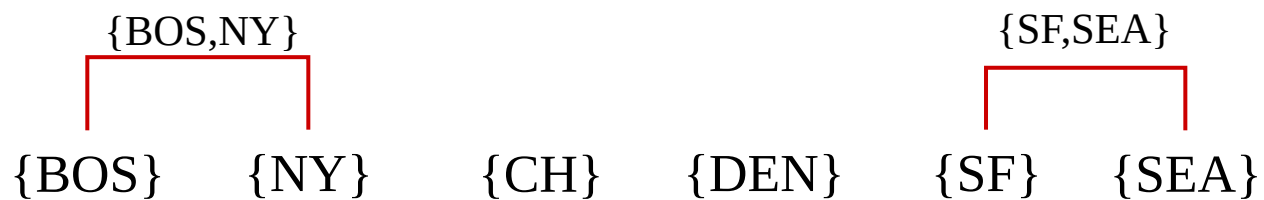
k=5





	BOS	NY	CHI	DEN	SF	SEA
BOS	0	206	963	1949	3095	2979
NY		0	802	1771	2934	2815
CHI			0	966	2142	2013
DEN				0	1235	1307
SF					0	808
SEA						0

Single linkage

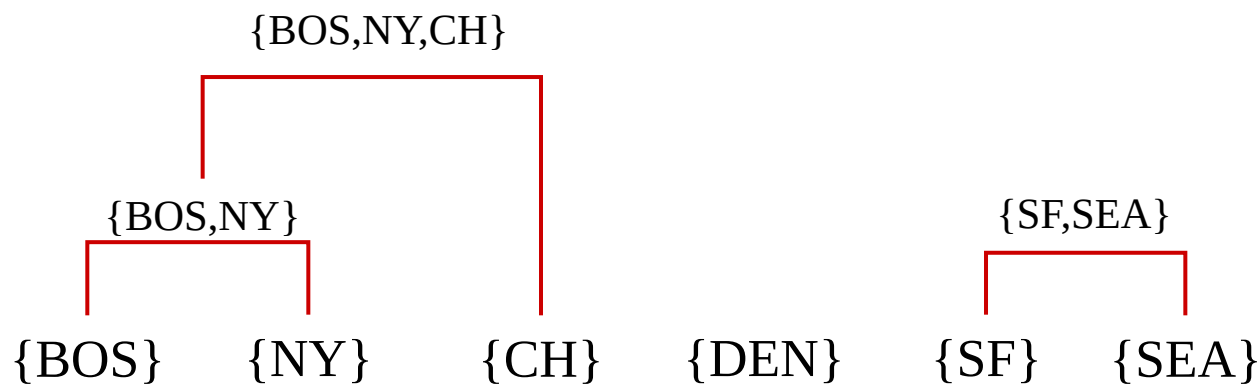
 $k=4$ 

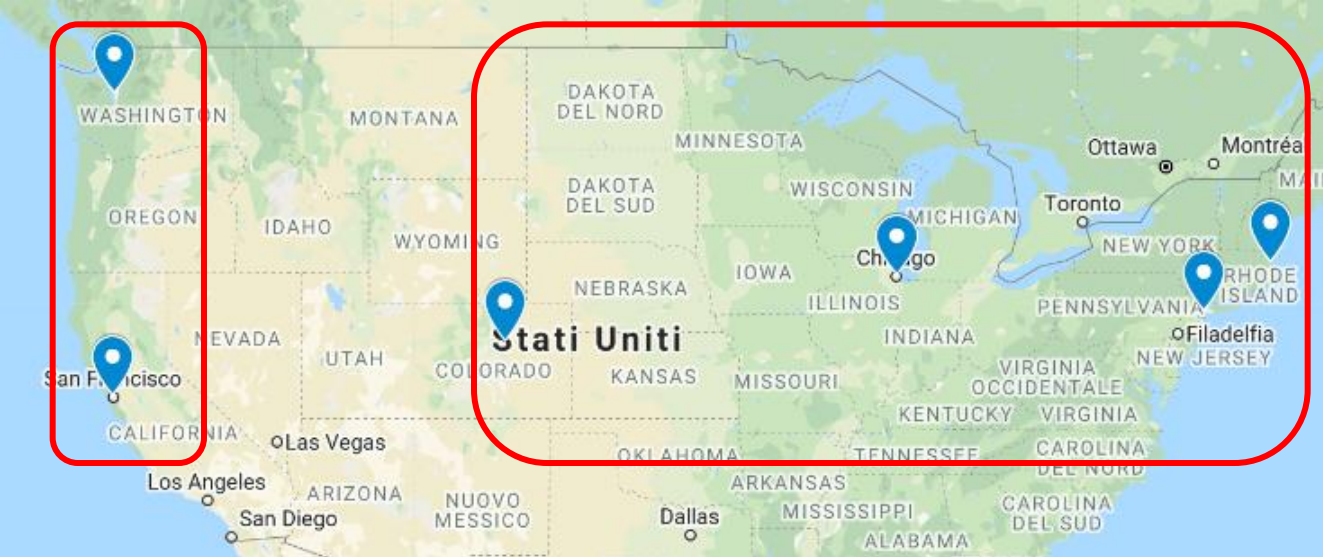


	BOS	NY	CHI	DEN	SF	SEA
BOS	0	206	963	1949	3095	2979
NY		0	802	1771	2934	2815
CHI			0	966	2142	2013
DEN				0	1235	1307
SF					0	808
SEA						0

Single linkage

$k=3$

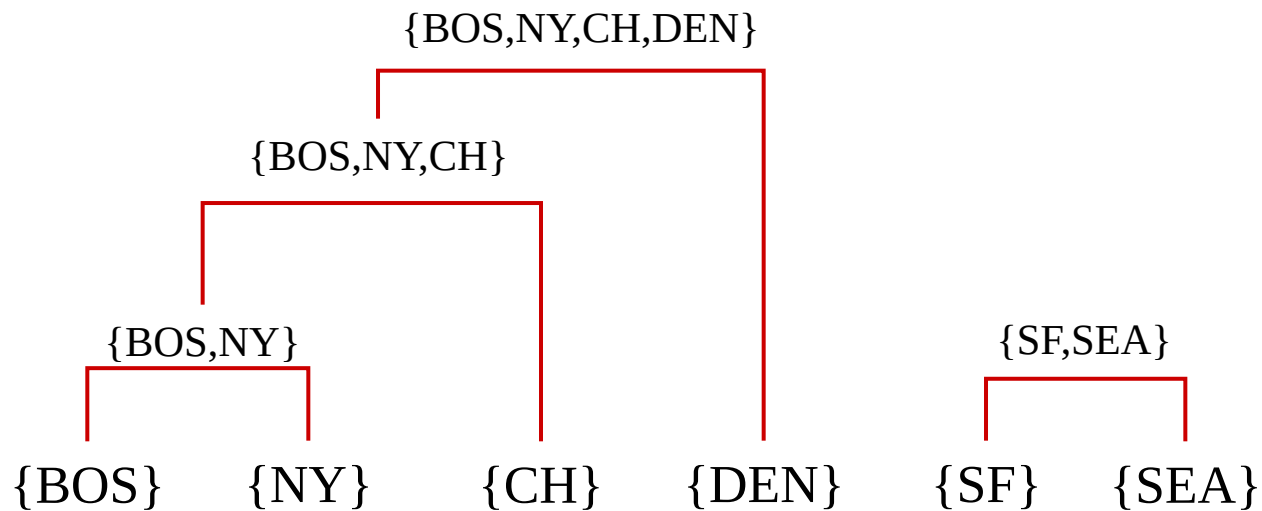


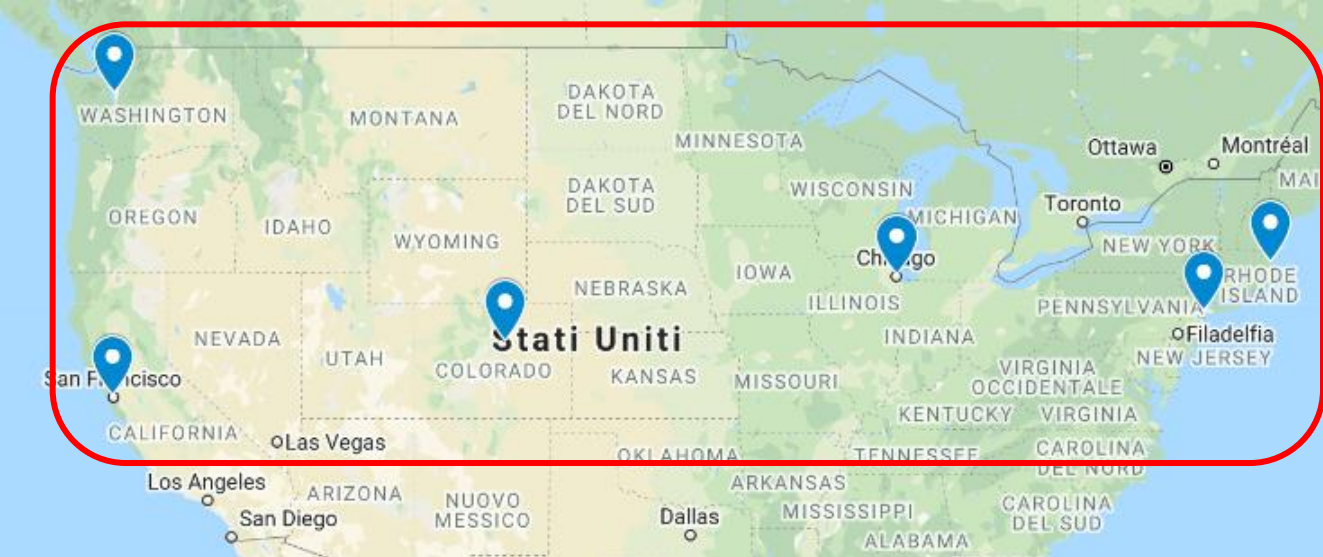


	BOS	NY	CHI	DEN	SF	SEA
BOS	0	206	963	1949	3095	2979
NY		0	802	1771	2934	2815
CHI			0	966	2142	2013
DEN				0	1235	1307
SF					0	808
SEA						0

Single linkage

$k=2$

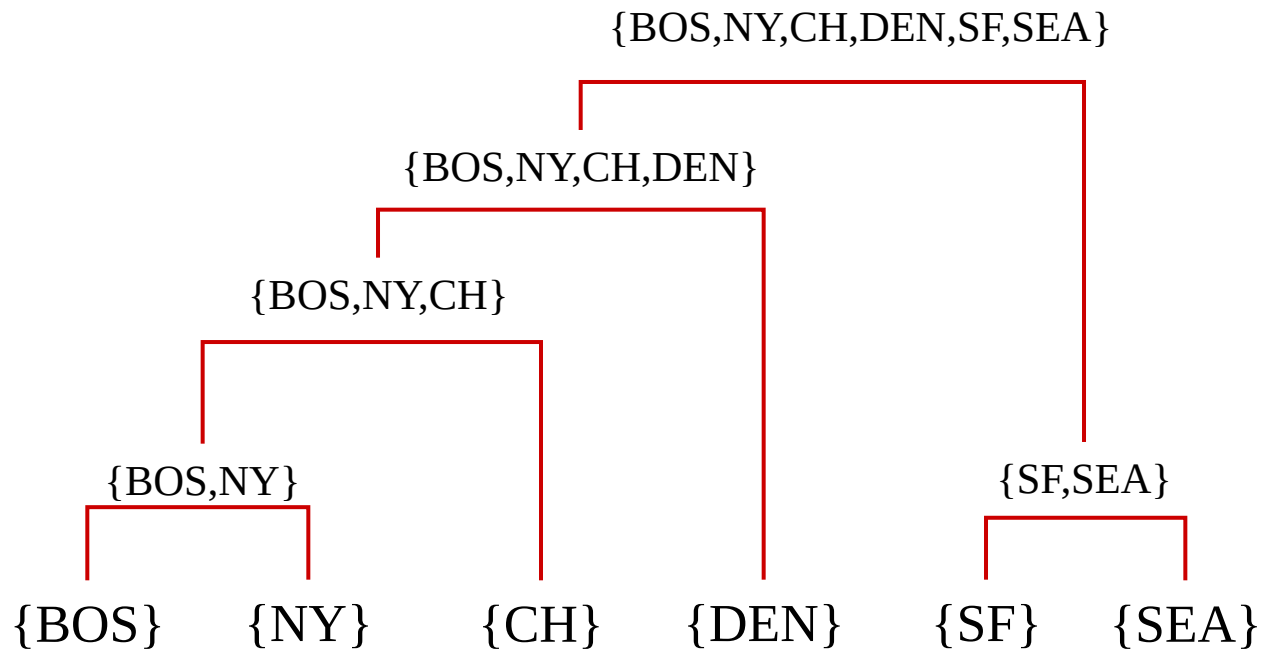




	BOS	NY	CHI	DEN	SF	SEA
BOS	0	206	963	1949	3095	2979
NY		0	802	1771	2934	2815
CHI			0	966	2142	2013
DEN				0	1235	1307
SF					0	808
SEA						0

Single linkage

$k=1$

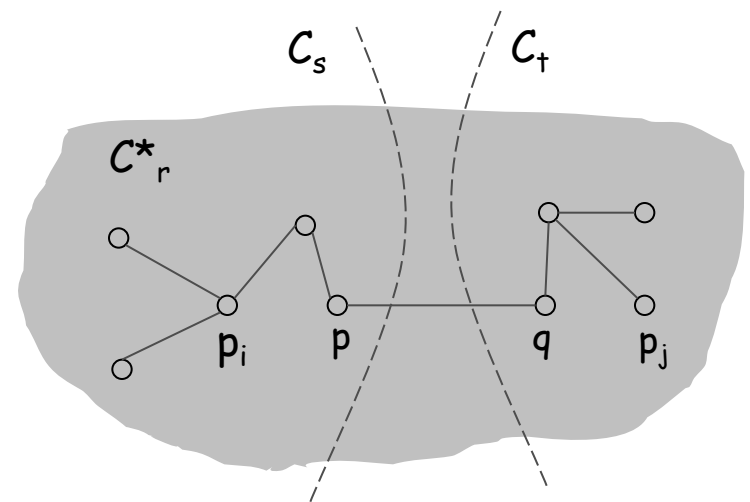


Greedy Clustering Algorithm: Analysis

Theorem. Let C^* denote the clustering $C^*_1, \dots, C^*_k$ formed by deleting the $k-1$ most expensive edges of a MST. C^* is a k -clustering of max spacing.

Pf. Let C denote some other clustering $C_1, \dots, C_k$.

- The spacing of C^* is the length d^* of the $(k-1)^{\text{st}}$ most expensive edge of the MST.
- Since $C^* \neq C$, there must exist p_i, p_j in the same cluster in C^* , say C^*_r , but in different clusters in C , say C_s and C_t .
- Some edge (p, q) on p_i - p_j path in C^*_r spans two different clusters in C .
- All edges on p_i - p_j path have length $\leq d^*$ since Kruskal chose them.
- Spacing of C is $\leq d^*$ since p and q are in different clusters. ▀



Extra Slides

MST Algorithms: Theory

Deterministic comparison based algorithms.

- $O(m \log n)$ [Jarník, Prim, Dijkstra, Kruskal, Boruvka]
- $O(m \log \log n)$. [Cheriton-Tarjan 1976, Yao 1975]
- $O(m \beta(m, n))$. [Fredman-Tarjan 1987]
- $O(m \log \beta(m, n))$. [Gabow-Galil-Spencer-Tarjan 1986]
- $O(m \alpha(m, n))$. [Chazelle 2000]
- $O(T^*(m, n)) = O(m \alpha(m, n))$. [Pettie-Ramachandran 2000]

Holy grail. $O(m)$.

Notable.

- $O(m)$ randomized. [Karger-Klein-Tarjan 1995]
- $O(m)$ verification. [Dixon-Rauch-Tarjan 1992]

